

Skill 9 os

SESSION 17

Task 1: To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings

Code:

```
rohita@DESKTOP-1UHU58C: ~  
GNU nano 8.7.1  
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
#include <sys/wait.h>  
#include <string.h>  
  
#define MAX_JOBS 10  
  
typedef struct {  
    int job_id;  
    pid_t pid;  
    char command[100];  
    int active;  
} Job;  
  
Job jobs[MAX_JOBS];  
int job_count = 0;  
  
void update_jobs() {  
    int status;  
  
    for (int i = 0; i < job_count; i++) {  
        if (jobs[i].active) {  
  
            pid_t result = waitpid(  
                jobs[i].pid,  
                &status,  
                WNOHANG  
            );  
  
            if (result == jobs[i].pid) {  
                jobs[i].active = 0;  
            }  
        }  
    }  
}  
  
void list_jobs() {  
    update_jobs();  
  
    printf("\n===== ACTIVE JOBS =====\n");  
    printf("JOB ID\tPID\tSTATUS\t\tCOMMAND\n");  
  
    int found = 0;
```

```
rohita@DESKTOP-1UHU58C: ~  
GNU nano 8.7.1  
  
    for (int i = 0; i < job_count; i++) {  
  
        if (jobs[i].active) {  
  
            printf("[%d]\t%d\tRUNNING\t\t%s\n",  
                jobs[i].job_id,  
                jobs[i].pid,  
                jobs[i].command);  
  
            found = 1;  
        }  
    }  
  
    if (!found)  
        printf("No active jobs.\n");  
}  
  
int main() {  
  
    char command[100];  
  
    printf("=====\n");  
    printf("        ACTIVE JOB LISTING\n");  
    printf("=====\n");  
  
    while (1) {  
  
        printf("\n$ ");  
        fflush(stdout);  
  
        fgets(command, sizeof(command), stdin);  
        command[strcspn(command, "\n")] = '\0';  
  
        if (strcmp(command, "exit") == 0)  
            break;  
  
        if (strcmp(command, "jobs") == 0) {  
            list_jobs();  
            continue;  
        }  
  
        int len = strlen(command);  
  
        /* Detect background command */  

```

```
rohita@DESKTOP-1UHU58C: ~
GNU nano 8.7.1
    if (len > 0 && command[len - 1] == '&') {

        command[len - 1] = '\\0';

        while (strlen(command) > 0 &&
            command[strlen(command) - 1] == ' ') {
            command[strlen(command) - 1] = '\\0';
        }

        if (job_count >= MAX_JOBS) {
            printf("Maximum job limit reached.\\n");
            continue;
        }

        pid_t pid = fork();

        if (pid < 0) {
            perror("fork");
            continue;
        }

        if (pid == 0) {

            execlp("sh", "sh", "-c", command, NULL);

            perror("exec");
            exit(1);
        }

        else {

            jobs[job_count].job_id = job_count + 1;
            jobs[job_count].pid = pid;
            strcpy(jobs[job_count].command, command);
            jobs[job_count].active = 1;

            printf("[%d] %d started in background\\n",
                jobs[job_count].job_id,
                pid);

            job_count++;
        }
    }

    else {
```

```
        else {
            printf("Use '&' to run a command in background.\\n");
            printf("Example: sleep 20 &\\n");
        }
    }

    return 0;
}
```

Output:

```

rohita@DESKTOP-1UHU58C:~$ nano s91.c
rohita@DESKTOP-1UHU58C:~$ gcc s91.c
rohita@DESKTOP-1UHU58C:~$ ./a.out
=====
ACTIVE JOB LISTING
=====

$ sleep 20 &
[1] 8709 started in background

$ sleep 30 &
[2] 8713 started in background

$ jobs

===== ACTIVE JOBS =====
JOB ID  PID    STATUS    COMMAND
[1]     8709   RUNNING   sleep 20
[2]     8713   RUNNING   sleep 30

$ exit
rohita@DESKTOP-1UHU58C:~$ █

```

Observation: The program successfully lists active background jobs with their **job ID, process ID, status, and command**. It continuously updates the job state using `waitpid()` and displays only currently active jobs.

Task 2: To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching

Code:

rohita@DESKTOP-1UHU58C: ~

GNU nano 8.7.1

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>
#include <string.h>
```

```
#define MAX_JOBS 10
```

```
typedef enum {
    RUNNING,
    STOPPED,
    COMPLETED,
    FOREGROUND
} State;
```

```
typedef struct {
    int job_id;
    pid_t pid;
    pid_t pgid;
    char command[100];
    State state;
} Job;
```

```
Job jobs[MAX_JOBS];
int job_count = 0;
```

```
/* Display jobs */
```

```
void show_jobs() {
    printf("\n----- JOB LIST ----- \n");

    for (int i = 0; i < job_count; i++) {
        printf("[%d] PID=%d PGID=%d %s\n",
            jobs[i].job_id,
            jobs[i].pid,
            jobs[i].pgid,
            jobs[i].command);
    }
}
```

```
/* Bring selected job to foreground */
```

```
void foreground_job(int job_id) {
    int index = -1;
```

rohita@DESKTOP-1UHU58C: ~

GNU nano 8.7.1

```
/* Identify target job */
```

```
for (int i = 0; i < job_count; i++) {
    if (jobs[i].job_id == job_id) {
        index = i;
        break;
    }
}
```

```
if (index == -1) {
    printf("Job [%d] not found.\n", job_id);
    return;
}
```

```
Job *job = &jobs[index];
```

```
printf("\nBringing job [%d] to foreground...\n", job->job_id);
```

```
/* Transfer terminal control to job */
tcsetpgrp(STDIN_FILENO, job->pgid);
```

```
/* Resume process */
if (kill(-job->pgid, SIGCONT) == -1) {
    perror("SIGCONT");
}
```

```
job->state = FOREGROUND;
```

```
printf("Terminal control transferred to PID %d\n",
    job->pid);
```

```
printf("Waiting for foreground job to finish...\n");
```

```
/* Wait for process */
int status;
```

```
waitpid(job->pid, &status, WUNTRACED);
```

```
/* Return terminal control to parent */
tcsetpgrp(STDIN_FILENO, getpgrp());
```

```
if (WIFEXITED(status)) {
    job->state = COMPLETED;
    printf("Job [%d] completed.\n", job->job_id);
}
```

```

rohita@DESKTOP-1UHU58C: ~
GNU nano 8.7.1
    printf("Job [%d] completed.\n", job->job_id);
}
else if (WIFSTOPPED(status)) {
    job->state = STOPPED;
    printf("Job [%d] stopped.\n", job->job_id);
}
}

/* Create background job */
void create_background_job(char *command) {

    if (job_count >= MAX_JOBS) {
        printf("Job table is full.\n");
        return;
    }

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return;
    }

    if (pid == 0) {

        /* Create new process group */
        setpgid(0, 0);

        /* Restore default signals */
        signal(SIGINT, SIG_DFL);
        signal(SIGTSTP, SIG_DFL);

        execlp("sh", "sh", "-c", command, NULL);

        perror("exec");
        exit(1);
    }

    /* Parent */
    setpgid(pid, pid);

    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;
    jobs[job_count].pgid = pid;
    strcpy(jobs[job_count].command, command);

```

```
rohita@DESKTOP-1UHU58C: ~
GNU nano 8.7.1
    jobs[job_count].pid = pid;
    jobs[job_count].pgid = pid;
    strcpy(jobs[job_count].command, command);
    jobs[job_count].state = RUNNING;

    printf("[%d] %d started in background\n",
           jobs[job_count].job_id,
           pid);

    job_count++;
}

int main() {

    char input[100];

    /* Ignore terminal control signals */
    signal(SIGTTOU, SIG_IGN);
    signal(SIGTTIN, SIG_IGN);

    printf("=====\n");
    printf("      FOREGROUND JOB SWITCHER\n");
    printf("=====\n");

    printf("\nCommands:\n");
    printf("  command &  -> Start background job\n");
    printf("  jobs       -> Display jobs\n");
    printf("  fg N       -> Bring job N to foreground\n");
    printf("  exit       -> Exit\n");

    while (1) {

        printf("\n$ ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strcmp(input, "jobs") == 0) {
            show_jobs();
        }
    }
}
```

```

/* fg command */
if (strncmp(input, "fg ", 3) == 0) {

    int job_id = atoi(input + 3);

    foreground_job(job_id);

    continue;
}

/* Background command */
int len = strlen(input);

if (len > 0 && input[len - 1] == '&') {

    input[len - 1] = '\0';

    while (strlen(input) > 0 &&
           input[strlen(input) - 1] == ' ') {
        input[strlen(input) - 1] = '\0';
    }

    create_background_job(input);
}
else {
    printf("Use '&' for background jobs.\n");
}
}

return 0;
}

```

Output:

```

rohita@DESKTOP-1UHU58C:~$ ./a.out
=====
      FOREGROUND JOB SWITCHER
=====

Commands:
  command &  -> Start background job
  jobs       -> Display jobs
  fg N       -> Bring job N to foreground
  exit       -> Exit

$ fg N
Job [0] not found.

$ jobs

----- JOB LIST -----

$ command &
[1] 8803 started in background

$ jobs

----- JOB LIST -----
[1] PID=8803 PGID=8803  command

$ fg N
Job [0] not found.

$ exit

```

Observation: The program successfully identifies the selected job and transfers terminal control to its process group using `tcsetpgrp()`. It resumes the process with `SIGCONT`, waits for completion, restores terminal control, and updates the job state accordingly.

SESSION 18

Task 1: To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior

Code:

```
rohita@DESKTOP-1UHU58C: ~
GNU nano 8.7.1 s93.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
void signal_handler(int sig){
    if(sig==SIGINT)
        printf("\nChild received SIGINT (Ctrl+C)\n");
    if(sig==SIGTERM)
        printf("\nChild received SIGTERM\n");
}
int main(){
    pid_t pid;
    sigset_t mask;
    printf("===== \n");
    printf("    SIGNAL BEHAVIOR ANALYSIS\n");
    printf("===== \n");
    sigemptyset(&mask);
    sigaddset(&mask,SIGINT);
    printf("\nParent PID: %d\n",getpid());
    printf("Parent Process Group: %d\n",getpgrp());
    pid=fork();
    if(pid<0){
        perror("fork");
        exit(1);
    }
    if(pid==0){
        printf("\n--- CHILD PROCESS ---\n");
        printf("Child PID: %d\n",getpid());
        setpgid(0,0);
        printf("Child Process Group: %d\n",getpgrp());
        signal(SIGINT,signal_handler);
        signal(SIGTERM,signal_handler);
        sigprocmask(SIG_BLOCK,&mask,NULL);
        printf("\nSIGINT is BLOCKED.\n");
        printf("Press Ctrl+C now...\n");
        sleep(5);
        sigset_t pending;
        sigpending(&pending);
        if(sigismember(&pending,SIGINT))
            printf("\nSIGINT is pending because it was blocked.\n");
        printf("\nUnblocking SIGINT...\n");
        sigprocmask(SIG_UNBLOCK,&mask,NULL);
        printf("SIGINT is now UNBLOCKED.\n");
        sleep(5);
    }
}
```

```

        sleep(5);
        printf("\nChild process completed.\n");
        exit(0);
    }else{
        printf("\nChild created successfully.\n");
        printf("Child PID: %d\n",pid);
        sleep(2);
        printf("\nParent is waiting for child...\n");
        wait(NULL);
        printf("Child process finished.\n");
    }
    return 0;
}

```

Output:

```

=====
      SIGNAL BEHAVIOR ANALYSIS
=====

Parent PID: 8876
Parent Process Group: 8876

Child created successfully.
Child PID: 8877

--- CHILD PROCESS ---
Child PID: 8877
Child Process Group: 8877

SIGINT is BLOCKED.
Press Ctrl+C now...

Parent is waiting for child...

Unblocking SIGINT...
SIGINT is now UNBLOCKED.

Child process completed.
Child process finished.

```

Observation: The program demonstrates signal delivery and signal masking by blocking SIGINT, detecting it as a pending signal, and delivering it after unblocking. It also demonstrates process groups and the behavior of terminal-generated signals such as Ctrl+C.

Task 2: To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verify Execution, Test Background Recovery

Code:

```

rohita@DESKTOP-1UHU58C: ~
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <string.h>
#define MAX_JOBS 10
typedef struct {
    int job_id;
    pid_t pid;
    char status[20];
} Job;
Job job_table[MAX_JOBS];
int job_count = 0;
void add_job(pid_t pid, const char *status) {
    if (job_count < MAX_JOBS) {
        job_table[job_count].job_id = job_count + 1;
        job_table[job_count].pid = pid;
        strcpy(job_table[job_count].status, status);
        job_count++;
    }
}
void update_status(pid_t pid, const char *status) {
    for (int i = 0; i < job_count; i++) {
        if (job_table[i].pid == pid)
            strcpy(job_table[i].status, status);
    }
}
void display_jobs() {
    printf("\nJob Table:\n");
    printf("Job ID\tPID\tStatus\n");
    printf("-----\n");
    for (int i = 0; i < job_count; i++)
        printf("%d\t%d\t%s\n", job_table[i].job_id, job_table[i].pid, job_table[i].status);
}
int main() {
    pid_t pid;
    int status;
    printf("Starting Job Control Demonstration\n");
    pid = fork();
    if (pid < 0) {
        perror("fork");
        exit(EXIT_FAILURE);
    }
    if (pid == 0) {

```

```

    }
    if (pid == 0) {
        printf("\nChild process started. PID = %d\n", getpid());
        printf("Child is stopping itself using SIGSTOP...\n");
        kill(getpid(), SIGSTOP);
        printf("Child resumed successfully!\n");
        for (int i = 1; i <= 5; i++) {
            printf("Child running: step %d\n", i);
            sleep(1);
        }
        printf("Child process completed.\n");
        exit(0);
    }
    add_job(pid, "Running");
    printf("\nParent created child process.\n");
    display_jobs();
    waitpid(pid, &status, WUNTRACED);
    if (WIFSTOPPED(status)) {
        update_status(pid, "Stopped");
        printf("\nChild process has been stopped.\n");
        display_jobs();
        printf("\nSending SIGCONT to resume the job...\n");
        if (kill(pid, SIGCONT) == 0) {
            update_status(pid, "Running");
            printf("SIGCONT sent successfully.\n");
        } else {
            perror("SIGCONT");
            exit(EXIT_FAILURE);
        }
        display_jobs();
    }
    waitpid(pid, &status, 0);
    if (WIFEXITED(status)) {
        update_status(pid, "Completed");
        printf("\nChild process completed normally.\n");
    }
    display_jobs();
    printf("\nJob recovery test completed successfully.\n");
    return 0;
}

```

Output:

```

rohita@DESKTOP-1UHU58C:~$ ./a.out
Starting Job Control Demonstration

Parent created child process.

Job Table:
Job ID  PID    Status
-----
1       8946   Running

Child process started. PID = 8946
Child is stopping itself using SIGSTOP...

Child process has been stopped.

Job Table:
Job ID  PID    Status
-----
1       8946   Stopped

Sending SIGCONT to resume the job...
SIGCONT sent successfully.

Job Table:
Job ID  PID    Status
-----
1       8946   Running
Child resumed successfully!
Child running: step 1
Child running: step 2
Child running: step 3
Child running: step 4
Child running: step 5
Child process completed.

Child process completed normally.

Job Table:
Job ID  PID    Status
-----
1       8946   Completed

Job recovery test completed successfully.

```

Observation: The stopped child process was successfully resumed using SIGCONT, and its status was updated from **Stopped** → **Running** → **Completed**.